n8watch

Night Network Watch

Real-time IP and network host monitoring with a web dashboard, historical metrics, configurable alert rules, and email notifications.

System Guide · Version 1.0 · Node.js 24.0

1 · System Overview

n8watch is a lightweight, self-hosted network monitoring system that continuously pings a configurable list of hosts, stores historical results in a local SQLite database, and pushes live metrics to a React web dashboard via WebSocket.

Key Features

- **Real-time ICMP monitoring** — sends configurable ping cycles (count, interval, timeout) to every target.
- **Web dashboard** — live status grid with latency, jitter, packet loss, and uptime badges.
- **Historical data** — SQLite store with configurable retention (default 90 days); CSV export.
- **Alert engine** — JavaScript expression conditions (e.g. `packet_loss > 20`) with per-rule cooldowns and per-target scoping.
- **Email notifications** — HTML-formatted SMTP alerts with deduplication via cooldown timers.
- **Live config reload** — edit `config.yaml` while running; targets and rules are applied within one second.
- **WebSocket push** — all connected browsers receive updated metrics immediately after each ping cycle.
- **Multi-interface support** — bind outgoing pings to a specific NIC per target.
- **User-defined targets** — add temporary ad-hoc hosts from the dashboard without editing config.
- **Electron desktop app** — optional standalone application for Windows, macOS, and Linux.

Architecture at a Glance

Layer	Technology	Role
Backend	Node.js · Express	REST API, WebSocket server, ping engine, alert engine, scheduler
Database	SQLite (better-sqlite3)	Persistent storage for targets, ping results, alerts
Frontend	React 18 · Vite · Tailwind CSS	Dashboard SPA (served as static files from `client/dist`)
Process mgr	PM2	Production process supervision, log rotation, system boot startup
Desktop	Electron (optional)	Wraps the server + SPA in a native application window

2 · Prerequisites & Installation

Prerequisites

Requirement	Version	Notes
Node.js	**24.0** (v18+ minimum)	nodejs.org — the setup scripts install v24 LTS automatically if Node.js is absent
npm	Bundled with Node.js	No separate install needed
`ping`	System binary	Pre-installed on Linux, macOS, Windows. On Debian/Ubuntu: `sudo apt install iputils-ping`
Git	Any recent version	Required for the one-line install scripts

> **Node.js v24 LTS** is strongly recommended. The automated setup scripts (`setup.sh` / `setup.ps1`) will install it automatically when Node.js is not already present.

One-Line Install (Linux / macOS) — Recommended

Clone the repository and run interactive setup in a single command:

```
# Using curl
curl -fsSL https://raw.githubusercontent.com/revul93/n8watch/main/install.sh | bash

# Using wget
wget -qO- https://raw.githubusercontent.com/revul93/n8watch/main/install.sh | bash
```

The installer clones the repo into an `n8watch/` directory then calls `setup.sh`.

One-Line Install (Windows — PowerShell)

```
Invoke-Expression (Invoke-WebRequest `
  -Uri "https://raw.githubusercontent.com/revul93/n8watch/main/install.ps1" `
  -UseBasicParsing).Content
```

Manual Installation

Step 1 — Clone the repository

```
git clone https://github.com/revul93/n8watch.git
cd n8watch
```

Step 2 — Run the interactive setup script

```
# Linux / macOS
bash setup.sh

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -File setup.ps1
```

The setup script performs the following steps automatically:

1. Checks for Node.js v24 LTS and installs it (via `nvm` or the platform package manager) if missing.
2. Installs backend Node.js dependencies (`npm install --omit=dev`).
3. Installs and builds the React frontend (`cd client && npm install && npm run build`).
4. Detects real network interfaces on the host and writes them into `config.yaml`; falls back to copying `config.example.yaml` unchanged.
5. Optionally installs PM2 globally, starts the application, saves the process list, and configures system-boot auto-startup.

Step 3 — Manual setup (alternative to the script)

```
# Backend dependencies
npm install

# Frontend build
cd client && npm install && npm run build && cd ..

# Configuration
cp config.example.yaml config.yaml
# → edit config.yaml with your targets and SMTP settings
```

> **After installation**, open `config.yaml` and set your target IP addresses and SMTP credentials before starting the application.

3 · Configuration (config.yaml)

All settings live in `config.yaml` at the project root. The file is watched at runtime — save your changes and they are applied within one second without restarting the process.

3.1 General Settings

```
general:
  ping_interval: 30        # Seconds between ping cycles (1–59)
  ping_count: 5            # ICMP packets sent per cycle
  ping_timeout: 5          # Timeout in seconds per ping attempt
  data_retention_days: 90  # Days to keep historical ping results
```

Key	Default	Description
`ping_interval`	30	Seconds between each full ping sweep of all targets
`ping_count`	5	Number of ICMP packets sent to each host per cycle
`ping_timeout`	5	Per-packet timeout in seconds
`data_retention_days`	90	Historical ping results older than this are automatically purged

3.2 Network Interfaces (optional)

```
interfaces:
  - name: "eth0"           # OS interface name (used with ping -I)
    alias: "Primary LAN"  # Human-readable label shown in the dashboard
    ipv4: "192.168.1.100" # IPv4 address bound to this interface
```

When present, the dashboard shows a drop-down in the *Add Target* panel and displays a teal interface badge on each target card. The setup script auto-detects interfaces on the host.

3.3 System-Defined Targets

```
targets:
  - name: "Google DNS"
    ip: "8.8.8.8"
    group: "DNS Servers"        # Optional grouping / filter label
    interface: "eth0"           # Optional: binds outgoing ping to this NIC
  - name: "Core Router"
    ip: "192.168.1.1"
    group: "Local Network"
```

Field	Required	Description
`name`	Yes	Display name in the dashboard and reports
`ip`	Yes	IPv4 address or resolvable hostname
`group`	No	Label used to filter targets in the dashboard
`interface`	No	OS interface name; must match an entry in the `interfaces` section

3.4 Server

```
server:
  port: 3000          # HTTP port (default 3000)
  host: "0.0.0.0"     # Bind address (0.0.0.0 = all network interfaces)
  # ssl:              # Uncomment to enable HTTPS
  #   enabled: true
  #   cert: "/etc/ssl/certs/n8watch.crt"
  #   key:  "/etc/ssl/private/n8watch.key"
```

3.5 Alerts & Email Notifications

```
alerts:
  email_notifications: true   # Set false to disable all email alerts

  smtp:
    host: "smtp.gmail.com"
    port: 587
    secure: false             # true = port 465 (TLS); false = port 587 (STARTTLS)
    user: "you@gmail.com"
    pass: "app-password"      # Use an app-specific password, not your account passwo
    from: "n8watch <monitor@yourdomain.com>"
    to:
      - "admin@yourdomain.com"

  rules:
    - name: "Host Down"
      condition: "packet_loss == 100"
      severity: "critical"    # critical | warning
      cooldown: 300           # Min seconds between repeated alerts for the same targ
      # targets:              # Omit or leave empty to apply rule to ALL targets
    - name: "High Latency"
      condition: "avg_latency > 200"
      severity: "warning"
      cooldown: 600
      targets:                # Restrict to specific targets by name, IP, or numeric
        - "Core Router"
```

Alert Rule Condition Variables

Variable	Type	Description
`is_alive`	0 or 1	Whether the host responded
`packet_loss`	0–100 (%)	Percentage of lost packets
`avg_latency`	ms	Average round-trip time
`min_latency`	ms	Minimum round-trip time
`max_latency`	ms	Maximum round-trip time
`jitter`	ms	(max − min) / 2
`packets_sent`	integer	Total packets sent in the cycle
`packets_received`	integer	Total packets received

Example conditions:

```
condition: "packet_loss == 100"   # Host completely unreachable
condition: "packet_loss > 20"     # More than 20% packet loss
condition: "avg_latency > 200"    # Latency exceeds 200 ms
condition: "jitter > 50"          # Jitter exceeds 50 ms
condition: "is_alive == 0"        # Host not responding
```

4 · How the System Works

4.1 Ping Engine & Scheduler

On startup, `server/scheduler.js` creates a `node-cron` job that fires every `general.ping_interval` seconds. Each tick triggers `server/ping-engine.js`, which pings all active targets in parallel. Results (min/avg/max latency, jitter, packet loss, alive status) are written to the `ping_results` SQLite table and immediately broadcast to all connected WebSocket clients.

4.2 Alert Engine

After each ping cycle, `server/alert-engine.js` evaluates every alert rule against the fresh metrics for each in-scope target. If a condition evaluates to `true` and the rule's cooldown timer has not yet elapsed, an alert record is written to the `alerts` table, a WebSocket `alert` event is pushed to the dashboard, and (if SMTP is configured) an HTML email notification is dispatched.

4.3 Live Config Reload

`server/config.js` uses Node.js `fs.watch` to monitor `config.yaml`. When a change is detected, the file is re-parsed and the application:

1. Re-syncs the target list in the database (adds new entries, removes deleted ones).
2. Restarts the ping scheduler with the (possibly new) interval.
3. Reinitialises the email service and alert engine with updated rules.
4. Broadcasts a `config_reloaded` WebSocket event so connected browsers can refresh.

4.4 System-Defined vs User-Defined Targets

System-Defined Targets

Targets listed in the `targets:` section of `config.yaml` are **system-defined**. They are loaded on startup, kept in sync on every live config reload, and persist as long as they appear in the file. They are stored in the database with `is_user_target = 0`.

User-Defined Targets

Any host added via the dashboard's *Add Target* panel (or via `POST /api/targets/user-targets`) is a **user-defined target**. These are useful for quick, ad-hoc monitoring without

editing config. They are stored with `is_user_target = 1`, are temporary by default, and can be removed individually from the dashboard or via `DELETE /api/targets/user-targets/:id`.

Feature	System-Defined	User-Defined
Defined in `config.yaml`	✅ Yes	❌ No
Persists across restarts	✅ Yes	⚠️ Temporary
Live-reloaded with config	✅ Yes	❌ No
Added via dashboard / API	❌ No	✅ Yes
Subject to alert rules	✅ Yes	✅ Yes
Manually removable from dashboard	❌ No	✅ Yes

4.5 WebSocket Events

Connect to `ws://<host>:3000` to receive real-time events:

Event	Emitted when
`ping_result`	After each full ping cycle — contains fresh metrics for all targets
`alert`	An alert rule fires for a target
`config_reloaded`	`config.yaml` is saved and reloaded

5 · Running the Application

5.1 Development (Node.js directly)

```
# Start the backend (serves the pre-built frontend from client/dist)
npm start
# or
node server/index.js

# Hot-reload frontend during UI development (separate terminal)
cd client
npm run dev    # Vite dev server on http://localhost:5173 — proxies API to :3000
```

Open `http://localhost:3000` (or the configured host/port) in your browser.

5.2 Production with PM2

PM2 is the recommended process manager — it restarts the app automatically, rotates logs, and can start on system boot.

```
# Install PM2 globally (one-time)
npm install -g pm2
```

npm run shortcuts

Command	What it does
`npm run pm2:start`	Start n8watch under PM2 using `ecosystem.config.js`
`npm run pm2:stop`	Stop the running process
`npm run pm2:restart`	Hard restart (brief downtime)
`npm run pm2:reload`	Graceful reload (zero-downtime restart)
`npm run pm2:logs`	Tail real-time application logs
`npm run pm2:status`	Show process status and uptime
`npm run pm2:save`	Persist the current process list
`npm run pm2:startup`	Generate and run the system-boot startup command

Direct PM2 commands (equivalent)

```
pm2 start ecosystem.config.js   # Start
pm2 stop n8watch                 # Stop
pm2 restart n8watch              # Hard restart
pm2 reload n8watch               # Zero-downtime reload
pm2 logs n8watch                 # Tail logs
pm2 status                       # Process list
pm2 monit                        # Real-time monitor
pm2 save && pm2 startup          # Enable auto-start on reboot
```

Log files

Logs are written to:

- `logs/out.log` — standard output
- `logs/error.log` — standard error

5.3 Windows (without PM2)

```
REM Option A — double-click or CMD
start.cmd

REM Option B — Node.js directly
node server\index.js

REM Option C — PM2 (recommended)
npm run pm2:start
```

6 · Database Maintenance Scripts

n8watch ships two maintenance scripts in `scripts/`. Always stop the application before running either script to avoid database conflicts.

6.1 flush-data.js — Clear Rows

Deletes rows from the database without removing the database file itself. Useful for freeing disk space or resetting metrics between test runs.

```
# Via npm shortcut (flushes ping results only — default)
npm run db:flush

# Via npm shortcut (flushes everything — ping results, alerts, and all targets)
npm run db:flush:all

# Direct invocation with flags
node scripts/flush-data.js --ping-results   # ping results only (default)
node scripts/flush-data.js --alerts         # alert records only
node scripts/flush-data.js --ping-results --alerts  # both
node scripts/flush-data.js --all            # ping results + alerts + targets
node scripts/flush-data.js --all --yes      # non-interactive (skip confirmation)
```

Flag	What is deleted
(none) or `--ping-results`	All rows in `ping_results`
`--alerts`	All alert records
`--ping-results --alerts`	Ping results *and* alert records
`--all`	Ping results, alerts, and all targets
`--yes`	Suppresses the confirmation prompt

6.2 nuke-db.js — Complete Wipe & Recreate

Deletes the database *file* entirely (including WAL and SHM sidecar files) and recreates an empty schema from scratch. Auto-increment counters are reset; no data survives.

```
# Via npm shortcut (interactive — requires typing YES)
npm run db:nuke

# Non-interactive (CI / automated scripts)
node scripts/nuke-db.js --yes
```

> **Warning:** `nuke-db.js` is irreversible. All ping history, alert records, and target entries are permanently deleted. After nuking, start the application normally — targets from `config.yaml` will be reloaded on the next ping cycle.

7 · Web Dashboard

Open `http://<host>:3000` after the application is running.

Page	URL	Description
Dashboard	`/`	Live grid of all monitored hosts — status, latency, jitter, packet loss, uptime
History	`/history`	Paginated historical ping results with date/target filters and CSV export
Alerts	`/alerts`	Active and historical alerts, acknowledgement, and configured rules viewer

Adding a Temporary Target

1. Click **Add Target** on the Dashboard page.
2. Enter a display name, IP address / hostname, and (optionally) select an interface from the drop-down.
3. Click **Add** — the target appears in the grid and is pinged in the next cycle.
4. Remove it with the **×** button on its card (user-defined targets only).

8 · API Quick Reference

All endpoints are prefixed with `/api` and return JSON.

Method	Endpoint	Description
GET	`/api/targets`	List all targets with latest metrics
GET	`/api/targets/:id/metrics`	Aggregated metrics for a target
GET	`/api/targets/:id/ping-results`	Ping history for a target
POST	`/api/targets/user-targets`	Add a temporary user-defined target
DELETE	`/api/targets/user-targets/:id`	Remove a user-defined target
GET	`/api/alerts`	List alert events
POST	`/api/alerts/:id/acknowledge`	Acknowledge an alert

GET	`/api/dashboard`	Summary stats (uptime, avg latency, active alerts)
GET	`/api/export`	Export metrics to CSV
GET	`/api/interfaces`	List interfaces from `config.yaml`

n8watch · Night Network Watch · Licensed under GNU GPL v3.0 · github.com/revul93/n8watch